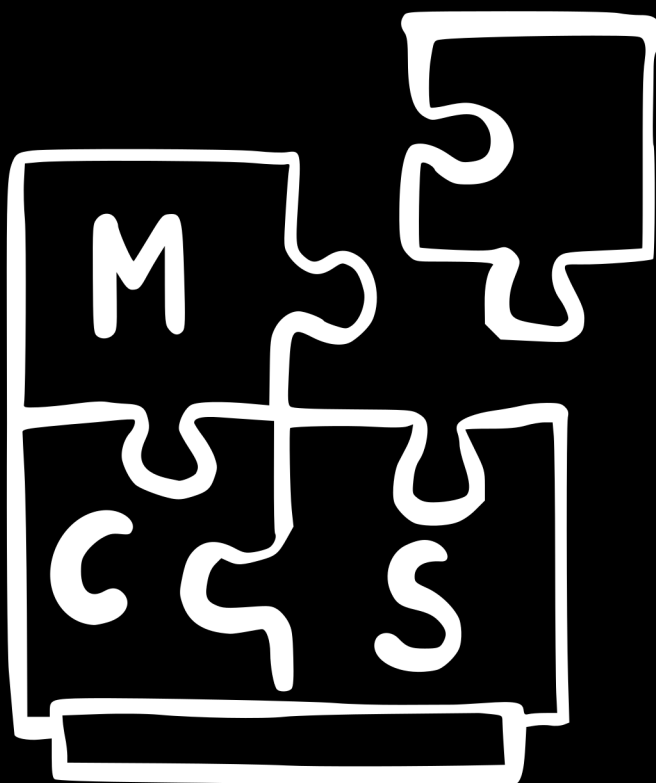




MODUCARD SYSTEM 3 HANDBOOK

MODULAR MECHANICA 2026

MODULAR MECHANICA 2026

MCS 3

MODUCARD SYSTEM

HANDBOOK

Contents

1	Fork	1
1.1	Dlaczego tak chaotycznie	1
1.2	Sekcje	1
1.3	Prezentacja	1
1.4	Język	1
1.5	Struktura	1
1.6	I reszta	1
2	Primer -> Zaczynij tutaj	3
2.1	Co i po co - Część #1 "Co"	3
2.1.1	Duży ModuCard a MiniModuCard	3
2.1.2	Naprawa i wytrzymałość	3
2.1.3	Uniwersalność	3
2.1.4	ModuCard zdjęcia	4
2.1.5	MiniModuCard zdjęcia	5
2.2	Co i po co - Część #2 "Po co"	6
2.2.1	Modularność	6
2.2.2	Naprawa przez wymienialność	6
2.2.3	Modularność jako organiczny rozrost	6
2.2.4	Podział na mniejsze elementy jako prostota	6
2.3	Co i po co - Część #3 "Co ale konkrety"	7
2.3.1	To samo, ale bardziej łopatologicznie	7
3	Skróty i nazwy własne	9
4	Książka elektroniczna	11
4.1	MiniModuCard - zasada działania	11
4.1.1	Overview	11
4.2	MiniModuCard - Kontroler	12
4.2.1	Jakie złącza	12
4.2.2	Komunikacja z hatami	12
4.2.3	Komunikacja z dużym ModuCardem	12
4.2.4	Programowanie	12
4.2.5	Jaki proc jest włożony	12
4.2.6	Zasilanie	13
4.2.7	Rysunek MMS3	13
4.3	MiniModuCard - Hat	13
4.3.1	Złącza	13
4.3.2	Modularność	13
4.3.3	Krawędzie	13
4.3.4	Rysunek hata	14
4.4	MiniModuCard - Łączenie hatów	14
4.5	Wybieranie hatów	14

4.5.1	Jak to działa	14
4.5.2	Wybieranie	14
4.5.3	Przesunięcie	15
4.5.4	Przykład	15
4.6	MiniModuCard - Interfejsy i protokoły komunikacyjne	15
4.6.1	MMS3 a haty	15
4.6.2	MMS3 a główny komputer ModuCard	16
4.6.3	Programowanie i debugowanie	16
4.7	MiniModuCard - AltMode i stopnie integracji	17
4.7.1	O co chodzi z rzędem AltMode	17
4.7.2	A z hatem AltMode?	17
4.8	MiniModuCard - Przykładowe MMS-y i haty	17
4.8.1	MMS3	17
4.8.2	Haty	18
4.9	MiniModuCard - Budżet mocy	19
4.10	MiniModuCard - Wymagania dla hata	19
4.11	MiniModuCard - Wymagania dla MMS-a	21
4.12	MiniModuCard - Projektowanie hatów	21
4.12.1	Co musisz zrobić	21
4.12.2	Co musisz wiedzieć	21
4.12.3	Templatka	22
4.12.4	Setup programów	23
4.12.5	Jak robić schematic/PCB	24
4.12.6	Pobieranie symboli i footprint'ów - pasywki	24
4.12.7	Pobieranie symboli i footprint'ów - IC	24
4.12.8	Parę moich komentarzy	25
4.12.9	Dodatkowe źródła do obczajenia	26
4.13	MiniModuCard - Pinout złączy	26
4.13.1	ChainBus + AltMode	26
4.13.2	ChainBus	27
4.13.3	ChainLink	28
4.13.4	XT60	29
4.13.5	JST-XH	29
4.13.6	I2C	30
4.13.7	Programowanie MCU na hatach	30
4.14	Duży ModuCard - Zasada działania	30
4.15	Duży ModuCard - Typy Kart	30
4.16	Duży ModuCard - Rola Backplane'a	30
4.17	Duży ModuCard - Przykładowe Karty	30
4.18	Duży ModuCard - Bilans Mocy	31
4.19	Duży ModuCard - Wymagania dla Kart	31
4.20	Duży ModuCard - Wymagania dla Backplane'a	31
4.21	Duży ModuCard - Projektowanie kart	31
4.22	Duży ModuCard - Pinout złączy	31
4.23	Pomiędzy systemami - Karta adapter i ChainLink	31
5	Książka programisty	33
5.1	Wstęp	33

6	Książka mechaniczna	35
6.1	Wymiary	35
6.1.1	Duży ModuCard	35
6.1.2	MiniModuCard - MMS3 (Mikrokontroler)	35
6.1.3	MiniModuCard - Hat	35
7	Książka system integrator	37
8	Książka ecosystem maintainer	39

1 Fork

1.1 Dlaczego tak chaotycznie

ogólnie potrzebuję pisać i mieć gdzie pokazać nowym ludziom jak to działa. Jeśli będą rzeczy w różnych językach, pomieszane, powtórzone itd to sorka. Potrzeba żeby ktoś napisał jak to działa faktycznie, zrobił zdjęcia a potem się to rzuci do Chata "popraw to. make no mistake".

1.2 Sekcje

Wstępnie planowaliśmy podzielić handbooka na kilka książek, jedna dla elektronika jedna dla programisty etc. Ale mi się będzie łatwiej pisać w jednym pliku, więc here we are.

1.3 Prezentacja

Kiedyś zrobiłem prezentację o ModuCardzie. Przerzucam wszystko co ważne w niej tutaj, w szczególności to co techniczne. Jeśli jej szukasz, to wszystko jest w tej książce jak na razie.

1.4 Język

Spoko byliby pisać dokumentację po angielsku, bo to poważny system a angielski to poważny język. Maybe another day.

1.5 Struktura

Zaczynij czytać od Primera, tam jest ogólny opis systemu. W rozdziałach ModuCard - zasada działania masz trochę więcej szczegółów, a potem w kolejnych rozdziałach masz już wszystkie szczegóły. Znowu to samo powtarzam w rozdziałach projektowanie kart/hatów. Jak nie załapiesz, to zapytaj się kogoś z koła.

1.6 I reszta

Aktualnie książka jest do użytku wewnętrznego KoNaR-u, ale możecie na spokojnie ją udostępniać komu trzeba. Czasem będę rzucił inside joke'i albo nazwiskami jak coś.

Fork jest mój, Wojtko Szafrńskiego (@themuse061 na Discordzie), a siedzę głównie w elektronice. Ona będzie najdokładniej opisana, mechanika też git powinna być, ale programowanie będzie tak sobie.

Oh, i możesz szukać "Jak projektować hat'a 4.12" albo "jak projektować Karte 4.21"

2 Primer -> Zaczynij tutaj

2.1 Co i po co - Część #1 "Co"

ModuCard to system dwóch standardów do tworzenia projektów robotycznych. Można w nim zrobić roboty, które wykonują skomplikowane czynności (chodzenie, regulacja procesów technologicznych, wizja komputerowa i inne).

Jest wymyślony w taki sposób, żeby zminimalizować ilość pracy włożoną w projektowanie i naprawianie robotów. Realizowane jest to przez używanie takich samych elementów w wielu miejscach. Te elementy są zgodne ze standardem, więc łatwo je zaprojektować oraz łatwo **do nich** zaprojektować mechanikę i programowanie. Czytaj: co tylko się da, ma takie same wymiary i złącza.

2.1.1 Duży ModuCard a MiniModuCard

- ModuCard (nazywany też dużym ModuCardem) -> Duże karty wpinane do backplate'a jak karta graficzna do płyty głównej. Na kartach dużego ModuCarda są bardziej wymagające peryferia, jakieś kamery, duże zasilacze, całe podsystemy. ModuCard to mózg całego systemu -> ma w sobie główny komputer systemu, analizuje dane, decyduje co zrobić dalej i steruje MiniModuCardem.
- MiniModuCard -> Haty wpinane do MMS-a. MMS to płytka z mikrokontrolerem i zasilaczem 5V. Na hatach są proste układy, które robią jedną rzecz, np. sterowniki silników, czujniki jakości powietrza, sterowniki taśm LED, przekaźniki. MiniModuCard to ręce i oczy systemu - wykonują pracę, raportują wartości czujników etc.

ModuCard i MiniModuCard komunikują się ze sobą dwiema liniami CAN. Taki protokół komunikacyjny wymyślony do samochodów. Jest spoko, bo wiesz, że jak wyślesz jakąś paczkę danych, to na pewno dojdzie do adresata, ma priorytety wiadomości i inne. Te same linie CAN idą przez duży i mały ModuCard.

2.1.2 Naprawa i wytrzymałość

Wszystkie elementy, z których złożony jest robot w ModuCardzie, są wymienne. Jak ci się zepsuje jakaś karta, to możesz ją po prostu wyjąć i włożyć nową, dokładnie taką samą. Te złącza, na które się wpinają, Artem (ten, co ogólnie wymyślił ten standard) zakosił z kolei, więc są całkiem odporne. Jak raz chłopaki wjechali łazikiem w ścianę, to sytuacja była analogiczna do Nokii 3310 spadającej na podłogę. MiniModuCard też ma podobny wąż, fajne złącza, modularność, łatwa wymiana i tak dalej.

2.1.3 Uniwersalność

Oh, i w ModuCardzie można nie tylko robić duże i poważne roboty. Nawet nie tylko roboty. Jak chcesz, to możesz robić instalację produkcyjną, jak w Hefajsosie (maszyna do przerobu odpadów w filament 3D) albo wziąć tylko MiniModuCarda i używać go jak Arduino. Wpinasz hata z przekaźnikiem, jakieś PWM-y i komunikację po UART i już możesz przełączać żarówki w smart home'ie.

I to były słowa. Co do zdjęć...

2.1.4 ModuCard zdjęcia

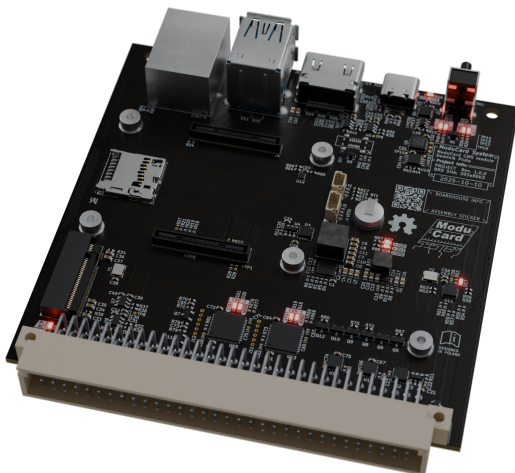


Figure 2.1: Pojedyncza karta ModuCard

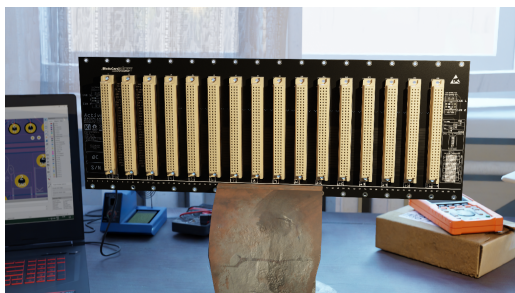


Figure 2.2: Backplane ModuCard. Do niego wpina się karty



Figure 2.3: Na zdjęciu są karty dużego ModuCarda wpięte do Backplane'a. Zdjęcie z robota Nomad

2.1.5 MiniModuCard zdjęcia

2.2 Co i po co - Część #2 "Po co"

Iii wracamy do słów.

2.2.1 Modularność

Cały ten system wyszedł z tego, że w sumie wszystkie roboty potrzebują tego samego - te same czujniki, te same silniki, te same procesory. Tylko w innych miejscach podpięte. Więc każdy robot miał nową płytkę PCB z tymi samymi elementami, ale z innymi złączami. Każda płytka mogła się zepsuć przy testowaniu. Weź teraz lutuj drugą albo projektuj to samo do nowego projektu, idzie się znużyć.

Żeby rozwiązać ten problem, w obu częściach systemu wszystko jest podzielone na mniejsze płytki, które robią tylko jedną rzecz. Kilka takich płytek łączysz ze sobą, żeby robiły kilka rzeczy i masz gotowego robota! Od razu możesz zamówić 5 płytek złutowanych, bo przydadzą się do innego projektu, więc nie szkoda kasy.

Jak raz zrobisz dobrą płytkę ze sterownikiem do silników, to idzie do wszystkich projektów. Nie musisz jej projektować ani programować od nowa do każdego, bo to jest to samo. I prawdopodobnie już ktoś kiedyś zaprojektował to, co potrzebujesz, a standard jest open source, to możesz sobie ukraść.

2.2.2 Naprawa przez wymienialność

Dodatkowo mega łatwo jest to wszystko naprawiać i modyfikować - jak coś ci się spali przy testowaniu robota, to wystarczy wyjąć zepsutą kartę "zasilacz 230" i włożyć zapa-sową. Są małe, więc szybki swap, kilka złączy przepiąć i na piwo. Plus tanio wychodzi, bo wymieniałeś tylko zasilacz, a nie całą płytę główną robota, więc możesz wziąć 2 piwa zamiast jednego (;

2.2.3 Modularność jako organiczny rozrost

A co do modyfikowania, to jak nagle się okaże, że potrzebujesz np. jeszcze jednego serwa w projekcie albo czegoś, to zamiast projektować od nowa "ROBOT MOTOR MAIN BOARD", która już działa, to wystarczy dołożyć nowy moduł (hata/kartę), który toysteruje. Nie musisz wiedzieć wszystkiego, co potrzebujesz na samym początku. Możesz w połowie projektu uznać, że w sumie fajnie byłoby mieć czujniki odległości, bez których myślałeś, że się obejdziesz.

2.2.4 Podział na mniejsze elementy jako prostota

Dalej, łatwo to projektować. Masz gotowe szablony projektów, na których już są podstawowe elementy, obrysy płytek i tylko dodajesz swoją funkcjonalność, często tylko 1 układ. I taka mała i prosta płytka tanio wychodzi, więc jak ją źle zaprojektujesz, to whatever.

2.3 Co i po co - Część #3 "Co ale konkrety"

Duży ModuCard składa się z:

- Backplane'a, do którego wpięte są karty.
- Backplane podłącza zasilanie i komunikację do kart.
- Komunikacja jest w postaci 2x CAN, 1x USB i 4x GPIO.
- Są 3 typy kart:
 - 1. Karta "główny komputer" - steruje całym systemem (np. Raspberry Pi).
 - 2. Karta Zasilanie - zasila system, duh.
 - 3. Karta Użytkowa/Mikrokontrolerowa - na niej są rzeczy, które faktycznie coś robią - kamery, radary i inne duże poważne rzeczy.
- Duży ModuCard zwykle zajmuje się wysokim sterowaniem, ale może robić cokolwiek, zależy jakie karty do niego włożysz.

Teraz mały ModuCard. On składa się z:

- MMS3 MCU - Jedna płytką z mikrokontrolerem, który kontroluje haty.
- Hatów - Do 8 płytek wykonawczych. Są np. sterownikiem silników, przetwornikiem termopary etc.
- (myśl o hatach jak o shieldach do Arduino, ale można ich dać aż 8).
- MCU i haty komunikują się po SPI, I2C i UART takim dużym złączem goldpin z dwoma rzędami.
- MCU ma złącze ChainLink (złączka RJ45, ta ethernet), na którym jest USB i 2x CAN.

I teraz ważne: to złącze ChainLink MMS3 z 2x CAN i USB można podłączyć do backplane'a dużego ModuCarda. Dzięki temu Raspberry z dużego ModuCarda może jednocześnie rozmawiać z kartami dużego ModuCarda i MiniModuCardem. Dzięki temu cały system, duży i mały ModuCard są w idealnej harmonii, dopełniają siebie.

2.3.1 To samo, ale bardziej łopatologicznie

Masz dużą szafę sterowniczą (Backplane), do której montujesz duże mózgi i poważne układy (karty). Od tej szafy odchodzi kabel (ChainLink), który idzie do małych sterowników (MMS) rozrzuconych po całym robocie, które ruszają silnikami i odczytują przyciski.

Trzymam kciuki, że chociaż ogólnie łapiesz, z czym to się je. Powodzenia z resztą dokumentacji (;

3 Skróty i nazwy własne

Jest dużo różnych protokołów, płytek i wgl. Tu jest lista:

- MMS - W zależności od kontekstu:
 - Albo chodzi o MiniModuCard System.
 - Albo o płytke z mikrokontrolerem sterującą całą wieżą hatów. Ta, która po CAN-ie rozmawia.
 - Raczej jak chodzi o płytke z MCU to jest MMS3, a jak system to MMS.
 - Często też pisze Kontroler MiniModuCard jako MMS3.
 - Ogólnie nazwy są jeszcze płynne.
- Backplane -> płytka w dużym ModuCardzie, do której wpina się karty. Ma w sobie 2x CAN, 4x GPIO, 1x USB i zasilanie.
- ChainLink -> połączenie między MMS3 a backplanem dużego ModuCarda. Ma 2x CAN, 1x USB, 12V stby.
- ChainBus -> magistrala komunikacyjna między kontrolerem a hatami.
- Karta ModuCard:
 - Karta CPU.
 - Karta PSU.
 - Karta Mikrokontrolerowa/Użytkowa.
- MCU - Mikrokontroler.
- Wieża, drzewko MMS -> 1 lub więcej hatów połączonych z Kontrolerem MiniModuCard.
- Sznurek/gałąź MMS3 -> kilka MMS3 podłączonych ze sobą po ChainLink.

4 Książka elektroniczna

4.1 MiniModuCard - zasada działania

4.1.1 Overview

Jak masz dużego ModuCarda, to każda kartka ma swój MCU i gdybyś robił osobną kartę na każdy feature (sterownik silnika, kontroler LED-ów), to wyszłoby ci drogo, połowy kanałów byś nie użył i te wszystkie karty potrzebują dużo miejsca. + parę innych problemów.

Więc zrobiliśmy mniejszy system, MiniModuCard. Basically Arduino, do którego dopinasz shieldy, ale można dużo tych shieldów dawać i sterować tym po CAN-ie, który łączysz potem z ROS-em (Robot Operating System). Arduino w naszym wypadku to płytka kontroler, a shield to hat.

MiniModuCard - Hat

Na hacie zwykle masz 1-2 IC, które coś robią, np. sterownik krokówki. Wyjścia są wyprowadzone na złączki JST-XH na długiej krawędzi płytki, a wszystko co konfiguracyjne na krótką krawędź. Tanio i łatwo to zaprojektować. Niby mało, ale w jednym projekcie używasz kilku hatów, np. możesz zrobić przycisk włącz-wyłącz na hacie GPIO, dodać hata LED do wyświetlania statusu, hat protocol z konwerterem UART -> RS485 i już masz elektronikę do sterowania falownikiem dla silnika 230V. Chcesz utrzymywać haty proste z jedną funkcjonalnością, żeby łatwo było składać je ze sobą.

Dalej, haty bardzo łatwo dodawać i odejmować z projektu, więc możesz postawić połowę elektroniki na nogi i dopiero potem zacząć projektować resztę. Nie musisz robić reworku softu od zera jak w normalnych projektach. Możesz wziąć na szybko hata z innej części projektu na testy, zanim zamówisz więcej. Możesz zacząć projekt, nie wiedząc czego potrzebujesz i na bieżąco robić haty.

MiniModuCard - Kontroler

MCU komunikuje się z hatem po SPI, I2C, UART i może wybierać, do którego hata się podłącza. Możesz myśleć o tym, że wszystkie linie komunikacji są podłączone do każdego hata przez przełącznik, więc możesz je kompletnie odłączyć (IRL jest bus switch zamiast przełącznika), więc z punktu widzenia twojego hata masz swoje własne linie SPI i UART. Tylko jeśli na swoim hacie potrzebujesz więcej niż jedno SPI albo UART, to musisz kombinować, np. jak na hacie protocol albo thermocouple.

MCU zajmuje się kontrolą wszystkich rzeczy, które są na hacie, czyli rozmawia z IC i steruje nimi. (btw, możesz dać też co-procesor, czyli jakieś mniejsze MCU na hata, jeśli nie ma IC do tego, zob. hat relay). Jego zadaniem jest też komunikowanie się CAN-em z komputerem pokładowym dużego ModuCarda i raportowanie stanu czujników oraz sterowaniem silnikami i innymi efektorami.

MCU można programować przez port USB, ale też zdalnie przez ChainLink. (Można flashować MCU zdalnie z poziomu komputera pokładowego dużego ModuCarda)

To, i jak tak ci bardziej odpowiada, to możesz użyć MCU bez dużego ModuCarda i CAN-a, żeby mieć mikrokontroler z masą gotowych i obkodzonych kart rozszerzeń.

4.2 MiniModuCard - Kontroler

Kontroler MiniModuCarda to główna płytką tego systemu. Ona steruje hatami oraz raportuje do dużego ModuCarda.

4.2.1 Jakie złącza

MMS3 ma:

- Złącze ChainBus + AltMode (AltMode możesz ignorować zob. 4.7) do podłączania hatów.
- 2x złącze XT60 do zasilania.
- 2x złącze ChainLink do podłączenia do dużego ModuCarda.
- USB-C do debugowania i flashowania.

4.2.2 Komunikacja z hatami

Jak pisałem w zasadzie działania, MMS3 ma SPI, I2C i UART podłączone do każdego hata. Domyślnie są odłączone, ale przez piny ADDR może się podłączyć do dowolnego. Więcej o wybieraniu w 4.5.

4.2.3 Komunikacja z dużym ModuCardem

MMS3 raportuje i dostaje polecenia od dużego ModuCarda protokołem CAN. Podpięty jest przez złącza ChainLink. ChainLink zaczyna się na karcie adapter (zob. 4.23) i idzie do MMS3. I teraz, MMS3 ma dwie złączki ChainLink, więc można podłączyć kolejnego MMS3 przez tę drugą łączkę (Daisy-Chain). Dlatego to jest ChainLink.

4.2.4 Programowanie

MMS3 można programować albo przez złącze USB-C na nim, albo zdalnie przez ChainLink.

W obu wypadkach USB idzie na huba, który podpina się naraz do programatora, który jest na płytce, i do pinów USB mikrokontrolera, który jest na MMS3, więc możesz też użyć USB serial do debugowania.

Ale jak to dwa wejścia USB naraz?

Jest zrobiony USB switch który automatycznie przełącza między USB-C a ChainLink. Jak podepniesz USB-C to switch to wykrywa i przełącza proca na USB-C.

Basically USB-C ma priorytet nad ChainLink'iem.

Programowanie z ChainLink'a

Troche jank, ale w ChainLink'u USB nie jest do niczego podłączone. Żeby podłączyć jakiegoś MMS3 do USB musisz przełączyć taki przełączniczek na MMS3 żeby się dioda świeciła. Wtedy MMS3 jest gotowy do komunikacji przez ChainLink.

Tylko jeden MMS3 można na raz programować po USB na ChainLink'u (czyli w wiazce MMS3 na tylko jednym ta dioda może być zapalona bo nie będzie działać). I ofc jak jesteś podpięty po USB-C to ono ma priorytet

4.2.5 Jaki proc jest włożony

Kontroler MiniModuCard to nie jest konkretna płytką, ale tak samo jak hat — rodzina płytek. Możesz zrobić kontroler na ESP32, Atmedze albo czym chcesz. W kole głównie używamy MMS3 na STM32F405 zrobionego przez Artema.

4.2.6 Zasilanie

MMS3 zasilą wszystkie haty szyną 5V. Przez to musi mieć na sobie przetworcę VBUS 12V-48V na 5V 1A.

4.2.7 Rysunek MMS3

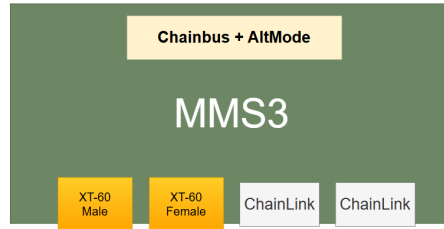


Figure 4.1: Widok kontrolera MMS3 z góry

4.3 MiniModuCard - Hat

Hat to ta płytki, która faktycznie coś robi w MiniModuCard. Do niej są podłączone czujniki, silniki i wszystko inne.

4.3.1 Złącza

- 2x ChainBus (jeden na dole, jeden na górze) do podłączenia hatów.
- 2x XT60 jeśli potrzebujesz więcej zasilania.
- 1x 4-pin złącze do flashowania EEPROMA.
- I złącza JST-XH, do których faktycznie podłączasz to, co musisz.

4.3.2 Modularność

Na jednym hacie jest tylko jedna rzecz - osobny hat na serwo, osobny na czujniki odległości itd. Dzięki temu każdy hat jest łatwo zaprojektować i łatwo je przenosić między projektami.

Do jednego MMS3 można podłączyć do 8 hatów. Jeśli potrzebujesz więcej, to musisz dodać nowy kontroler MiniModuCard ze swoją wieżą hatów.

4.3.3 Krawędzie

Mamy 4 krawędzie, powiedzmy NEWS jak kierunki świata:

- N, długa górna - tam jest ChainBus.
- E, krótka prawa - tam są diody LED, zworki konfiguracyjne i przyciski.
- W, krótka lewa - ta zostaje pusta.
- S, długa dolna - tam są złącza XT60 i JST-XH.

4.3.4 Rysunek hata

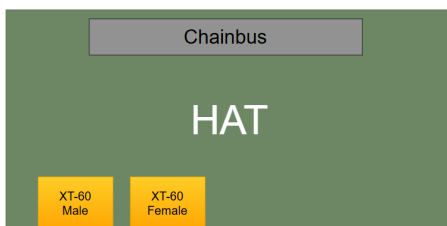


Figure 4.2: Widok hata z góry

4.4 MiniModuCard - Łączenie hatów

Na początku masz kontroler MMS. Do niego wpinasz kolejne haty. Kontroler ma złącze albo 3x16, albo 2x16 goldpin żeński THT. Do niego wpinasz hata. Ten pierwszy hat ma z dołu złącze 2x16 goldpin męski SMD, które wpinasz do MMS3. I teraz na pierwszym hacie na górze masz złącze 2x16 goldpin żeński SMD, do którego możesz wpiąć kolejnego hata.



Figure 4.3: 3 haty wpięte w Kontroler MMS3

- To żółte to złącze z kontrolera MMS3.
- A czarne to złącza hatów. Dolny jest męski, górny żeński.

4.5 Wybieranie hatów

Haty są podpięte przez bus switche, które odpinają i podpinają magistrale. Tak wygląda wybieranie:

4.5.1 Jak to działa

MMS3 MCU ma 8 pinów, ADDR1 do ADDR8. Służą do adresowania hatów. Poprzez wystawianie na odpowiednie HIGH albo LOW można wybierać haty.

4.5.2 Wybieranie

Bus switche na hatach mają odwróconą logikę. Jak dostają 3V3, to są wyłączone, a jak 0V, to podpinają hata do magistral. Każdy hat patrzy tylko na swoją linię ADDR1, a resztę przekazuje dalej.

4.5.3 Przesunięcie

Z MMS piny ADDR idą prosto na pierwszego hata. Ale z hatów idą już na skos - linia ADDR3 idzie na ADDR2, ADDR2 na ADDR1 itd. Linia wchodząca na ADDR1 jest używana przez hata i nieprzekazywana dalej.

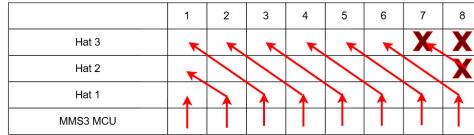


Figure 4.4: Tak wyglądają te połączenia hatów i MMS3

4.5.4 Przykład

MMS3 chce wybrać 3. hat.

- Zaczynamy z punktu wyjścia, wszystkie ADDR są na 3V3 (HIGH), czyli logiczne 0. Żaden hat nie jest wybrany.
- MMS3 ustawia ADDR3 na LOW.
- Pierwszy hat widzi, że ADDR3 jest LOW, reszta HIGH. Jego ADDR1 jest HIGH, czyli nie reaguje.
- Drugi hat widzi, że ADDR2 jest LOW, reszta HIGH. Jego ADDR1 jest HIGH, czyli nie reaguje.
- Trzeci hat widzi, że ADDR1 jest LOW, reszta HIGH. Jego ADDR1 jest LOW, czyli się aktywuje.
- MMS3 komunikuje się z hatem 3.
- MMS3 ustawia ADDR3 na HIGH, hat się wyłącza.
- I wracamy do punktu wyjścia, można aktywować kolejny hat.

To jest tylko komunikacja, tak to haty pracują w tle (np. czujniki temperatury mogą się kalibrować, silniki kręcić ze stałą prędkością etc).

	1	2	3	4	5	6	7	8
Hat 3	L	H	H	H	H	H	H	H
Hat 2	H	L	H	H	H	H	H	H
Hat 1	H	H	L	H	H	H	H	H
MMS3 MCU	H	H	L	H	H	H	H	H

Figure 4.5: To, o czym właśnie napisałem

4.6 MiniModuCard - Interfejsy i protokoły komunikacyjne

4.6.1 MMS3 a haty

MMS3 i haty komunikują się po SPI, I2C i UART przez złącze ChainBus.

I2C

Normalne I2C, ale każdy hat ma osobną pulę adresów. Adres 1010000 jest zajęty przez EEPROM (np. M24C64 z A0 A1 A2 zwartymi do masy). EEPROM ma dane identyfikacyjne (nazwa, UUID, hardware revision, software revision)

SPI

No, normalne SPI. Jak potrzebujesz mieć więcej slave'ów niż 1, to zabraknie ci Chip Select. Polecam dać ekspander GPIO po I2C albo zaimplementować expander na CH32V003, bo jest tańszy.

UART

UART. Pamiętaj, że podpinasz się do hatów tylko na chwilę, więc nie możesz cały czas nasłuchiwać RX. Chcesz wysłać coś po TX i dostać odpowiedź.

4.6.2 MMS3 a główny komputer ModuCard

MMS3 komunikuje się z komputerem pokładowym ModuCarda po ChainLink.

CAN

Są 2 linie CAN w ChainLink. Jakoś to gada. Problem programisty. Jak jesteś programistą, to masz problem.

USB

Jest 1 linia USB. Można się nią podłączyć do MMS3 i go programować po niej.

4.6.3 Programowanie i debugowanie

Więc hata da się debugować po USB-C.

USB hub

MMS3 ma wbudowanego huba USB i programator. Więc każdego MMS3 możesz debugować jednym kablem, bo masz USB hub -> Mikrokontroler (więc możesz użyć np. USB serial) i masz USB hub -> programator -> mikrokontroler (więc możesz wgrywać program przez to USB).

USB-C a ChainLink

Więc to jest opcja debugowania i flashowania lokalnego. Możesz też robić to samo z poziomu głównego komputera ModuCard przez ChainLink, bo do niego idzie USB przez ChainLink. MMS3 ma przełączniki USB, które automatycznie przełączają się między USB-C a ChainLinkiem. Normalnie jest do ChainLinka, ale jak się podłączysz kablem, to priorytet ma kabel.

Sznurek/gałąź MMS3

Każdy MMS3 ma 2 złącza ChainLink, więc możesz je daisy-chainować (łączyć pierwszy z kolejnym i tak dalej). Dzięki temu musisz wyprowadzić tylko jedno złącze ChainLink z ModuCarda i do tego możesz podłączyć ile chcesz MMS3.

Adnotacja: Chyba huby USB mają limit, ile ich można dać za sobą, albo signal integrity USB się spierdzieli. Czyli jest jakiś limit, jak będę wiedział jaki, to napiszę.

4.7 MiniModuCard - AltMode i stopnie integracji

4.7.1 O co chodzi z rzędem AltMode

MMS3 ma 3 rzędy pinów, a hat tylko 2. Wątek jest taki: mikrokontrolery zwykle mają więcej pinów niż potrzeba na sam ChainBus, więc te luźne wystawiliśmy na 3. rząd. Oto dlaczego:

Normalnie MMS3 używamy z ChainBusem + hatami, więc mamy wszystkie zalety modularności etc. Fajnie wszystko idzie, jak prototypujesz albo rozwijasz projekt. Ale gdybyś chciał bardziej zintegrować, pomniejszyć projekt, to możesz zrobić hata AltMode.

Pinout rzędu AltMode jest inny dla każdego MMS3.

4.7.2 A z hatem AltMode?

Jak już masz przetestowane układy i kod, to możesz swoje 8 hatów połączyć w jednego dużego. Teraz możesz użyć pinów AltMode i adresowania jako zwykłe GPIO i nowe magistrale komunikacyjne. Wtedy na jednego MMS3 wpinasz jedną dużą płytkę, która robi to samo co 8 hatów.

Już raczej jej nie będziesz przenosił między projektami, ale to jest tylko do finalizacji projektów. I dalej, możesz na tę samą płytkę dać samego MMS3 i mieć cały projekt na 1 PCB.

Zaleta od zaczęcia tak zaraz zamiast skipnięcia od razu do 1 PCB jest taka, że możesz wszystko stopniowo testować, łatwo modyfikować i tanio prototypować.

W kole nie używamy ani AltMode, ani stopniów integracji projektu 2 i 3.

Stopnie integracji

- Stopień 1 - MMS3 + 8 hatów ChainBus.
- Stopień 2 - MMS3 + 1 hat AltMode.
- Stopień 3 - Wszystko na tym samym PCB.

4.8 MiniModuCard - Przykładowe MMS-y i haty

4.8.1 MMS3

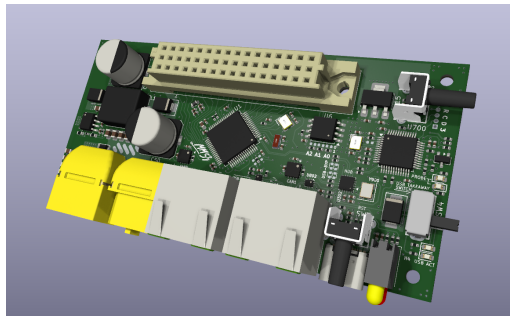


Figure 4.6: MMS3 STM32F405

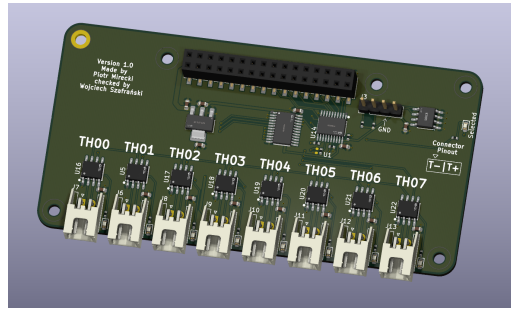


Figure 4.10: Hat thermocouple

4.9 MiniModuCard - Budżet mocy

Normalnie MMS3 zasilaasz ze złącza XT60 i to idzie na BRD VIN. MMS3 ma obowiązek zrobić z tego przetworke BUCK DC-DC na 5V co najmniej 1A. 12V Standby nie używajcie. I ChainBus za dużo nie puści prądu BRD VIN przez siebie, więc jak hat pobiera więcej prądu niż jakieś 185mA z niego, to dajcie XT60 na swojego hata. Ona może puścić max 60A.

Więc jak mamy np. 1A 5V, to jak dasz 8 hatów, to każdy może pociągnąć 125mA. Ale jak dasz tylko 2 haty, to już każdy może pociągnąć 500mA. A niektóre haty mogą ciągnąć mniej, jak mają np. same czujniki.

Na dole masz mniej więcej limity, staraj się ich nie przekraczać, a jak przekraczać to ostrożnie:

Nazwa	Napięcie	Sumaryczny prąd	prąd na jeden hat
5V	5.0 V	1.0 A	125 mA
12V stby	12.0 V	0.5 A	65 mA
BRD_VIN	12.0 V – 48.0 V	1.5 A	185 mA

Komponenty podłączone do linii BRD_VIN muszą być przystosowane do pracy z napięciem od 12V do **48 V**.

4.10 MiniModuCard - Wymagania dla hata

Tbh większość z tego jest już na templatce:

- Wymiary 100x50mm.
- Ma 6 otworów montażowych M2.5.
- Nie daje komponentów na keepout zone'ach (zob. sekcja mechaniczna).
- I komponenty tylko z przodu, nie wyższe niż XXXmm.
- Złącza XT60 są na długiej krawędzi.

- Złącza wyjściowe są na długiej krawędzi.
- Złącza wyjściowe mają raster 2.54mm i używamy JST-XH.
- Zworki konfiguracyjne są na prawej krótkiej krawędzi.
- Przyciski, LED-y też na prawej.
- lol lewa jest wolna.
- Złącze ChainBus w dobrym miejscu, z góry female i z dołu male.
- ChainBus przesuwa piny ADDR, reszty nie.
- Obsługuje napięcie na BRD VIN do 48V.
- Nie pobiera więcej prądu niż z budżetu mocy (kinda whatever).
- Podłącza ChainBus tylko do SPI, I2C, UART i nie odwala nic dziwnego z nimi.
- Nic dziwnego, między innymi sam nie zaczyna komunikacji, tylko grzecznie czeka na MMS3 aż zacznie gadać.
- Podłącza magistralę tylko kiedy ADDR1 = LOW.
- Ma EEPROM do identyfikacji po I2C o dobrym adresie i zgodnej komunikacji.
- Ma złącze I2C (3V3, GND, SCL, SDA) na prawej krawędzi, żeby łatwo flashować EEPROM.
- Ma diodę, która się świeci, jak hat jest wybrany.
- Ma pull-upy /downy na komunikacji (i ADDR1), żeby nie była floating jak hat jest odpięty.
- Jest w licencji TO-DO.
- Ma zdjęcie, BOM na GitHubie.
- Ma dokumentację na GitHubie.
- Ma soft na GitHubie.
- Ma tagi hashtag XXX na dole dokumentacji, żeby dało radę go wyszukać.
- Wszystkie złączki mają opisany pinout na silkscreenie.
- Nazwa, autor i rewizja hata jest opisana na silkscreenie.
- Duuużo diod, żeby się ładnie świeciło.
- Same złączki też opisane do czego są dużymi literami, żeby było ładnie.

4.11 MiniModuCard - Wymagania dla MMS-a

- Jakiś proc na nim.
- Złącza XT-60.
- Przetwornica VBUS -> 5V 1A.
- Przełączanie ADDR.
- Obsługa protokołów na ChainBus.
- Wymiary, złącza i otwory w dobrych miejscach.
- Przełączanie USB między złączką C a ChainLinkiem.
- USB hub na programator i USB z MCU.
- Obsługa ChainLink.
- 2x CAN.

4.12 MiniModuCard - Projektowanie hatów

4.12.1 Co musisz zrobić

Jeśli tu jesteś, to znaczy, że ktoś cię poprosił o zrobienie Hat'a do MiniModuCard'a. W tej książce jest cała dokumentacja systemu, ale postaram się streścić najważniejsze rzeczy, które musisz wiedzieć tutaj.

Zakładam, że dostałeś zadanie w stylu "zrób hat'a, który odbiera GPS", "który steruje silnikiem BLDC" albo "zrób hata z układem scalonym DRV17182". Jeśli się zgadza, to możesz czytać dalej.

4.12.2 Co musisz wiedzieć

Powinieneś przeczytać Primer (2), zasadę działania MiniModuCard'a (4.1) oraz sekcję o Hat'ach (4.3).

Pozwolę sobie założyć, że tego nie zrobiłeś, i na szybko jeszcze raz to opiszę.

Co to hat

Mamy system, w którym jest płytką z mikroprocesorem (np. STM32F405) i chcemy, żeby coś robiła. Żeby to było modularne, oddzieliliśmy procesor i układy wyjściowe na osobne płytki. Te osobne płytki z układami wyjściowymi to są "Hat'y" i ty takiego masz robić.

Mówiąc układ wyjściowy, mam na myśli np. właśnie sterownik silnika albo moduł GPS. Cokolwiek, co coś odczytuje albo steruje poza płytką.

Inaczej mówiąc, hat to tak jakby shield na Arduino.

Jak to działa

Procesor ma swoje piny (to, co podpinasz Arduino do modułów). One zmieniają swoje napięcia i sterują rzeczami wyprowadzonymi na złącze ChainBus. Złącze ma konkretne rzeczy na konkretnych pinach, żeby było łatwo wymieniać i projektować hat'y.

Te konkretne rzeczy na konkretnych pinach to I2C, SPI i UART. To są takie mega często stosowane protokoły komunikacyjne. Ich będziesz używać do sterowania układami scalonymi (IC) na hat'ie.

Twoje zadanie to znaleźć układ, który robi to, co chcesz (np. sterownik termopary) taki, żeby komunikował się po I2C, SPI albo UART, a nie jakimś dziwnym własnym protokołem. Jak już znajdziesz, to zapytaj się osoby, od której dostałeś zadanie, czy będzie pasował.

Jak masz ten układ wybrany, to musisz narysować w programie "KiCad", do czego jest podłączony. To jest schematic. Potem przenosisz go na fizyczną płytkę w tym samym programie. Wtedy masz PCB.

(Jeszcze jest system automatycznego wybierania Hat'ów przez piny ADDR, ale nie musisz się o nim martwić, bo już jest na templatce).

4.12.3 Templatka

Żeby było łatwiej projektować, to zrobiłem projekt z podstawowymi elementami hat'a na GitHubie, który możesz z'fork'ować i na nim projektować.

Link -> https://github.com/KoNaR-Hefajstos/MMS3_hat_templates

Tak wygląda otworzony w programie KiCad (musisz wejść w widok schematu/płytki)

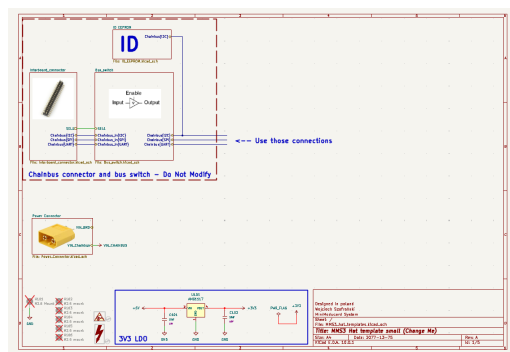


Figure 4.11: Widok schematic templatki

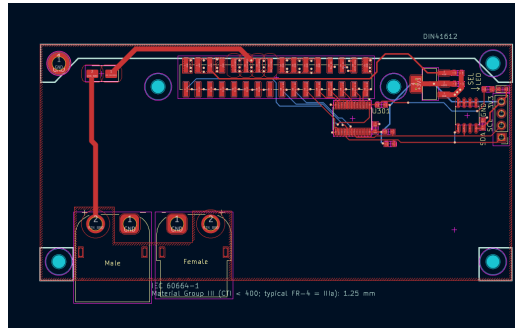


Figure 4.12: Widok płytki PCB templatki

(Zdjęcia mogą być stare, są z 18.07.26)

Noteworthy rzeczy o templatce

- Masz już zrobione złącze ChainBus, adresowanie i resztę
- Nie możesz rzucać Interboard connector, Bus switch, ID eeprom na schematic'u. One muszą zostać takie, jakie są
- Nie zmieniaj rozmiaru płytki PCB i nie ruszaj rozstawu śrub montażowych oraz złącz ChainBus.
- Możesz usunąć ze schematic'a i PCB złącze XT60, jeśli go nie potrzebujesz (ono jest, jak potrzebujesz dużo prądu)
- Na schematic'u możesz się podpiąć do I2C, SPI i UART, jak wychodzą z tej ramki, za bus switchem. Nie ruszaj nic w środku ramki
- Schematic jest podzielony na hierarchical sheet'y (np. Bus switch. Te prostokąty, możesz na nie dwa razy kliknąć, żeby zobaczyć do środka). Proszę też ich używaj <3
- O, i co do hierarchical sheet'ów (schematów hierarchicznych), to możesz zobaczyć, czy na innym hat'ie nie ma jakiegoś, który ci się przyda. Są opisane w README
- I2C, SPI i UART są w formie BUS connection. Zobacz w ID eeprom, jak wchodzi gruba niebieska linia, a w środku rozbija się na dwie małe zielone.

Basically musisz dodać swój układ i połączyć z jedną/dwoma/trzema protokołami komunikacyjnymi wychodzącymi z tej bordowej przerywanej ramki. I nie usuwać/przesuwać ważnych rzeczy.

Parę jeszcze rzeczy jest napisane w README template'a.

4.12.4 Setup programów

- Musisz zainstalować KiCad'a <https://www.kicad.org>
- I te pluginy do niego:
- Do generowania pliku BOM (Bill of materials) <https://github.com/openscopeproject/InteractiveHtmlBom>

- Do generowania plików produkcyjnych (tzw. gerber) <https://github.com/Bouni/kicad-jlcpcb-tools>
- Do pobierania footprint'ów komponentów z LCSC <https://github.com/uPesy/easyeda2kicad.py>
- Biblioteka podstawowych komponentów z JLCPCB assembly: <https://github.com/CDFER/JLCPCB-Kicad-Library>

4.12.5 Jak robić schematic/PCB

Pełne tłumaczenie od zera jest out of scope tego dokumentu (wierzę, że znajdziesz jakiś fajny poradnik na YT), ale

4.12.6 Pobieranie symboli i footprint'ów - pasywki

(symbol - element na schematic'u. Jakie ma połączenia. Footprint - element na PCB, jaki ma fizyczny kształt)

Zwykle zamawiamy PCB częściowo zlutowane (basic assembly) z JLCPCB. Trzeba używać komponentów od nich, żeby nie trzeba było ich później ręcznie wybierać przy zamawianiu. Dodatkowym bonusem jest to, że ładnie widać je na modelu 3D.

Elementy pasywne (pasywki - rezystory, kondensatory, LED-y etc) są już pobrane razem z pluginami. Musisz tylko dobre wybrać. Dobre, to znaczy z kategorii PCM_JLCPCB.

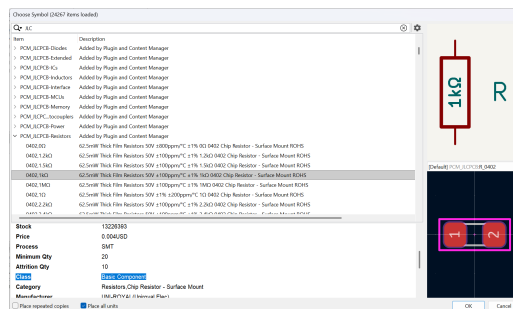


Figure 4.13: Widok z dodawania komponentów KiCad

Staraj się wybierać, jak możesz, komponenty "Basic Component". Za każdy inny komponent trzeba zapłacić dodatkowo 1.5\$ za załadowanie do maszyny pick and place. Zazwyczaj basic będą tylko podstawowe rezystory, kondensatory, diody (w tym LED), oscylatory i parę innych. Pełną listę możesz znaleźć, wchodząc na stronę <https://jlcpcb.com/parts> i odfiltrować tylko te basic. Najprawdopodobniej lutowane będą tylko te basic więc im więcej takich użyjesz tym mniej zabawy potem z kupowaniem i lutowaniem części.

Jak nie ma rezystora, którego potrzebujesz, to możesz połączyć 2/3 ze sobą. KiCad ma wbudowany do tego kalkulator.

4.12.7 Pobieranie symboli i footprint'ów - IC

Jeśli nie ma twojego komponentu automatycznie pobranego do KiCad'a (czyli większość nie-basic), musisz go ręcznie pobrać. Używamy do tego plugina easyeda2kicad.

A więc, procedura, której ja używam, wygląda tak:

- Znajdź swój komponent na <https://www.lcsc.com>

- Otwórz CMD (konsolę) w **folder_twojego_hat'a**/hardware/lib (możesz wejść do lib, kliknąć tam na górze, jak jest napisane, w jakim folderze jesteś (path) w pustym miejscu, tak żeby ci się podświetliło na niebiesko, wpisać "CMD" i dać Enter)
- Użyj komendy: **easyeda2kicad -lcsc_id <ID_komponentu_LCSC> -full -output ./<nazwa_folderu>**

```
C:\Users\Hase\Desktop\Pulpit\hola_naukowe\VolkarRobotics\..._PM53 Hat\PM53_hat_Meter-Source\Hardware\Lib\easyeda2kicad --
lcsc_id C5173369 -full -output ./I2C_GPII_expander
--
Easyeda2kicad.py v1.0.1 --
[INFO] Created kicad symbol for ID : C5173369
Symbol name : 9001-15481C08A_C5173369
Library path : I2C_GPII_expander.kicad_sym
[INFO] Created kicad footprint for ID : C5173369
Footprint name : CONN-TM_MINGTAT_9001-15481C08A
Footprint path: I2C_GPII_expander.pretty\CONN-TM_MINGTAT_9001-15481C08A.kicad_mod
[INFO] Created 3D model for ID : C5173369
3D model name : CONN-TM_MINGTAT_9001-15481C08A
3D model path (stl) : I2C_GPII_expander.3dshapes\CONN-TM_MINGTAT_9001-15481C08A.stl
3D model path (step) : I2C_GPII_expander.3dshapes\CONN-TM_MINGTAT_9001-15481C08A.step
C:\Users\Hase\Desktop\Pulpit\hola_naukowe\VolkarRobotics\..._PM53 Hat\PM53_hat_Meter-Source\Hardware\Lib\
```

Figure 4.14: Komenda do pobrania komponentu przez easyeda2kicad

- Powinno ci się pobrać plik **nazwa_folderu.kicad_sym** i foldery **nazwa_folderu.3dshapes** oraz **nazwa_folderu.pretty**. Jeśli ich nie masz, to w złym miejscu otworzyłeś CMD.
- Przenieś .kicad_sym oraz folder .3dshapes do folderu .pretty
- Otwórz swój projekt (dwa razy kliknij na .kicad_pro)
- Załaduj symbol -> wejdź w Symbol Editor, w lewym górnym wybierz File -> Add library i dodaj plik .kicad_sym jako "Add new library to project library table"
- Teraz załaduj footprint -> pójdź z powrotem do projektu, wejdź w Footprint Editor, i tak samo jak wcześniej, tylko wybierz folder .pretty
- Gotowe! Teraz możesz wejść do swojego projektu (widok schematic) i dodać świeżo zaimportowany komponent. Powinien mieć od razu footprint i model 3D.

4.12.8 Parę moich komentarzy

- Jak możesz, to daj diody LED do czego się da, żeby widać było, że coś się dzieje, jak płytka działa (zob. hat protocol)
- Jak skończysz robić PCB, to opisz wszystkie złącza, diody i co się da na silkscreenie (zob. hat HC-SR04-HX711)
- Pamiętaj podpisać, jaki to hat, która wersja i kto ją zrobił na płytce na silkscreenie
- W KiCadzie jest widok 3D, jak otworzysz PCB. Zobacz go sobie
- 1 / 1.5mm wielkości tekstu silkscreen to minimum, żeby było fajnie widać
- Używaj pasywek wielkości 0603
- Jeśli układ, który wybrałeś, ma jakieś piny interrupt, to je ignorujesz, zostaw je floating (do niczego niepodpięte). Jeśli ma jakieś piny konfiguracyjne, to daj je na zworki goldpin na boku płytki (jeśli trzeba je często przełączać - goldpin kątowny jest spoko) albo przez zworke lutowalną/rezystor na samej płytce (jeśli mniej często trzeba je przełączać). Tutaj musisz trochę pomyśleć, jak to musisz zrobić, żeby było dobrze. Zapytaj się kogoś, jak nie wiesz

- Jako że nie ma wyprowadzonych zwykłych pinów GPIO do hat'a, tylko same protokoły komunikacyjne, to może ci ich brakować. Jeśli będziesz ich potrzebował, to daj expander GPIO po I2C
- Robienie schematic'a (co do czego jest podłączone) nie jest trudne. Basically musisz wziąć jakiś "Typical design" z noty katalogowej układu (ang. datasheet) albo płytkę, którą już ktoś zrobił
- Jak robisz fork'a, to chcesz, żeby właścicielem repo była organizacja Konar'u (np. Hefajstos), a nie ty, bo jak jest na ciebie, to nie mam dostępu, żeby poprawić płytkę, jak ją sprawdzam
- Złącza wyjściowe (to, po co robisz płytkę) daj jako kątowe JST-XH 2.5mm/2.54mm (któreś z tych dwóch)
- Złącza konfiguracyjne (jeśli jakieś masz) daj na prawej krawędzi hat'a. Resztę rzeczy na dolną. Lewą zostaw pustą
- Jeśli szukasz układu scalonego i są tylko z własnym protokołem albo reszta jest droga, to możesz dać po drodze innego proca, np. CH32V003, który to przetłumaczy na I2C/SPI. Napisz do mnie albo innego ogarniętego elektronika, żeby ci to wytłumaczył
- Komponenty zamawiamy zwykle albo z TME, Mouser, albo LCSC. TME jest preferowane, ale jest drogo i mało komponentów

4.12.9 Dodatkowe źródła do obciążenia

Spoko miejsca do znalezienia układu i dokumentacji

- Moduły do Arduino
- <https://www.mikroe.com>

Spoko filmy/źródła o projektowaniu schematic'ów/PCB

- <https://www.youtube.com/@PhilsLab>
- <https://blog.poly.nomial.co.uk/2025-08-10-creating-high-quality-electronics.html>

Na zakończenie

To tyle z tłumaczenia. Jeśli jest coś, czego nie rozumiesz, myślisz, że można lepiej wytłumaczyć albo brakuje zdjęć, to proszę, **proszę** napisz mi co. Imo bardzo ważne jest, żeby ta część dokumentacji była zrozumiała, bo początkujący będą ją próbowali zrozumieć.

4.13 MiniModuCard - Pinout złączy

4.13.1 ChainBus + AltMode

Jest na MMS3.

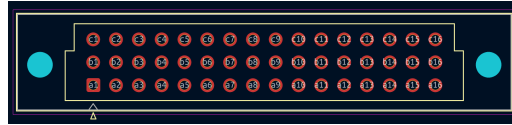


Figure 4.15: DIN 41612 pinout

Pin #	Row A (AltMode)	Row B (Address/Bus)	Row C (Power)
1	Altmode 1	ADDR 1	GND
2	Altmode 2	ADDR 2	+5V
3	Altmode 3	ADDR 3	+5V
4	Altmode 4	ADDR 4	GND
5	Altmode 5	ADDR 5	BRD_VIN
6	Altmode 6	ADDR 6	BRD_VIN
7	Altmode 7	ADDR 7	BRD_VIN
8	Altmode 8	ADDR 8	BRD_VIN
9	Altmode 9	ChainBus SDA	GND
10	Altmode 10	ChainBus SCL	GND
11	Altmode 11	ChainBus CS	GND
12	Altmode 12	ChainBus SCK	GND
13	Altmode 13	ChainBus MOSI	Reserved
14	Altmode 14	ChainBus MISO	Reserved
15	Altmode 15	ChainBus RX	+12V stby
16	Altmode 16	ChainBus TX	GND

4.13.2 ChainBus

Jest na każdym hacie.

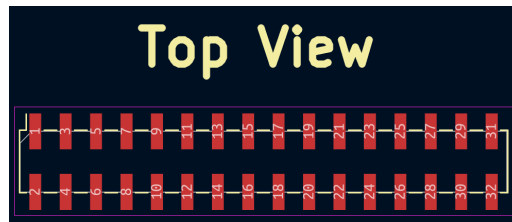


Figure 4.16: Złącza 2x16 SMD na górnej warstwie

Okay, wątek jest taki - ChainBus trzeba dać na górnej i dolnej warstwie. Wszystkie piny poza ADDR idą bezpośrednio z dolnej do górnej, a ADDR idą z przeplotem.

Table 4.1: ChainBus in, dolna warstwa, 2x16 smd male

Pin	Sygnal	Pin	Sygnal
1	GND	2	ADDR1
3	+5V	4	ADDR2
5	+5V	6	ADDR3
7	GND	8	ADDR4
9	VIN_ChainBus	10	ADDR5
11	VIN_ChainBus	12	ADDR6
13	VIN_ChainBus	14	ADDR7
15	VIN_ChainBus	16	ADDR8
17	GND	18	ChainBus SDA
19	GND	20	ChainBus SCL
21	GND	22	ChainBus CS
23	GND	24	ChainBus SCK
25	Pin_25	26	ChainBus MOSI
27	Pin_27	28	ChainBus MISO
29	+12V stby	30	ChainBus TX
31	GND	32	ChainBus RX

Table 4.2: ChainBus out, górna warstwa, 2x16 smd female

Pin	Sygnal	Pin	Sygnal
1	GND	2	ADDR2
3	+5V	4	ADDR3
5	+5V	6	ADDR4
7	GND	8	ADDR5
9	VIN_ChainBus	10	ADDR6
11	VIN_ChainBus	12	ADDR7
13	VIN_ChainBus	14	ADDR8
15	VIN_ChainBus	16	NC
17	GND	18	ChainBus SDA
19	GND	20	ChainBus SCL
21	GND	22	ChainBus CS
23	GND	24	ChainBus SCK
25	Pin_25	26	ChainBus MOSI
27	Pin_27	28	ChainBus MISO
29	+12V stby	30	ChainBus TX
31	GND	32	ChainBus RX

4.13.3 ChainLink

2x są na każdym MMS3 i są na kartach adapter.

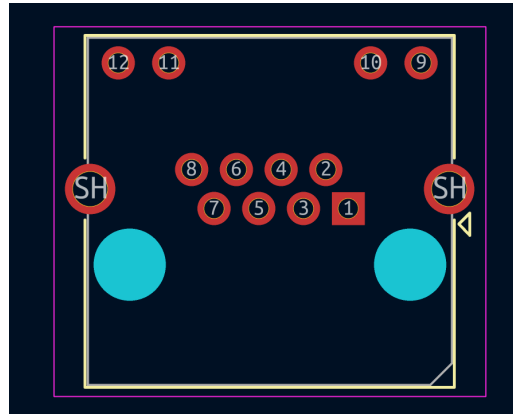


Figure 4.17: Złącze RJ45

Pin	Sygnal
1	CAN1.-
2	CAN1.+
3	GND
4	CAN2.-
5	CAN2.+
6	+12V stby
7	USB_MMS_.D-
8	USB_MMS_.D+

4.13.4 XT60

Zasilanie, pewnie wszędzie jest.

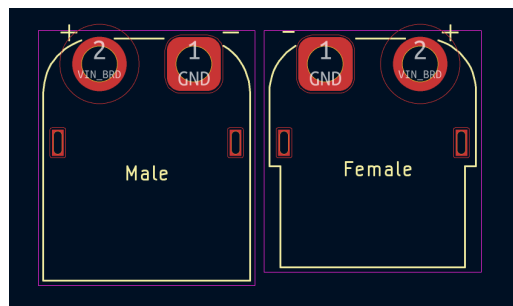


Figure 4.18: XT60 męska i żeńska

4.13.5 JST-XH

Wszystko idzie na JST-XH z rastrem 2.54 (same as goldpin), więc nie można dać uniwersalnego pinoutu.

Jedynie powiem, że preferowany jest GND na pinie 1.

4.13.6 I2C

Jak masz I2C na hatach, to to jest preferowany pinout:

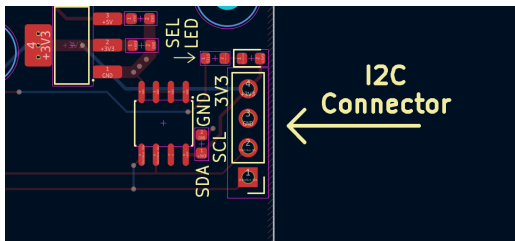


Figure 4.19: Złącze goldpin 1x4 z I2C na nim

4.13.7 Programowanie MCU na hatach

Jeśli masz MCU na hacie, to to są preferowane pinouty złącz do flashowania:

CH32V003

Jest podobne do WCH Link-E.

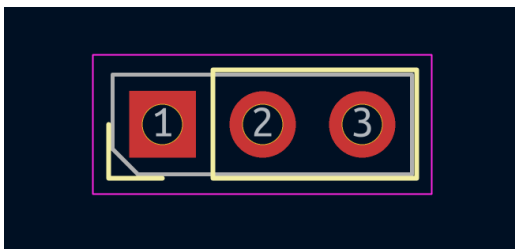


Figure 4.20: JST-XH 1x3

Pin	Sygnał
1	SWIO
2	GND
3	3V3

4.14 Duży ModuCard - Zasada działania

4.15 Duży ModuCard - Typy Kart

4.16 Duży ModuCard - Rola Backplane'a

4.17 Duży ModuCard - Przykładowe Karty

4.18 Duży ModuCard - Bilans Moc

4.19 Duży ModuCard - Wymagania dla Kart

4.20 Duży ModuCard - Wymagania dla Backplane'a

4.21 Duży ModuCard - Projektowanie kart

Sorka, To be done

4.22 Duży ModuCard - Pinout złączy

4.23 Pomiędzy systemami - Karta adapter i ChainLink

5 Książka programisty

5.1 Wstęp

Aktualnie nie mamy nic zaprogramowane. Powodzenia!

6 Książka mechaniczna

6.1 Wymiary

Jeśli jesteś mechanikiem zespołu, to najlepszą metodą na złapanie wymiarów jest poproszenie elektronika, żeby wyeksportował ci PCB do .step razem z komponentami.

Ta sekcja jest bardziej pod kątem utrzymania standardu.

Wymiary do śrub montażowych albo keepout zone'ów (to jest miejsce, gdzie elektronik nie może stawiać komponentów i ty możesz tam sobie coś zamodelować do złapania hata) - są symetryczne, więc paru wymiarów nie podałem.

6.1.1 Duży ModuCard

To Do

6.1.2 MiniModuCard - MMS3 (Mikrokontroler)

To Do

6.1.3 MiniModuCard - Hat

Odległość między hatem a hatem albo MMS3 a hatem to 1 złącze goldpin SMD męskie + wpięte w nie SMD żeńskie.

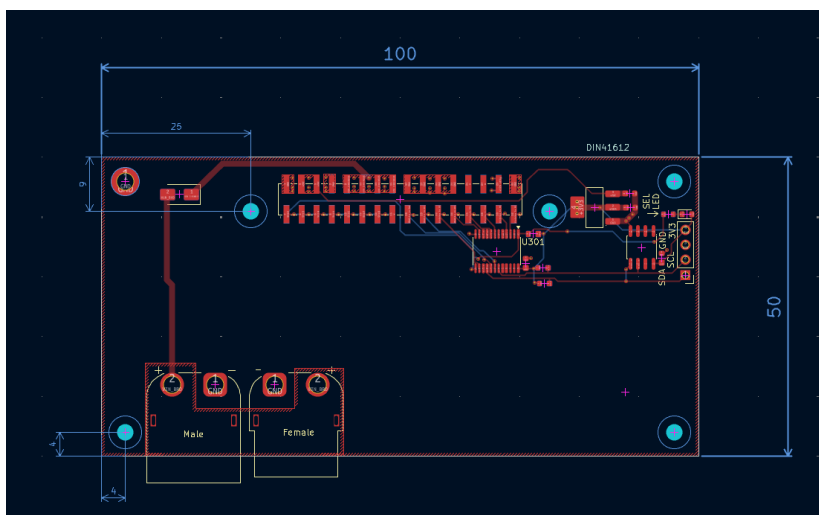


Figure 6.1: Wymiary płytki i otworów montażowych

Te złącza w lewym dolnym to XT60 i tñh nie muszą być co do milimetra w tym miejscu. Ważne, żeby były jak najbardziej w lewym dolnym i pierwsze męskie, a potem żeńskie.

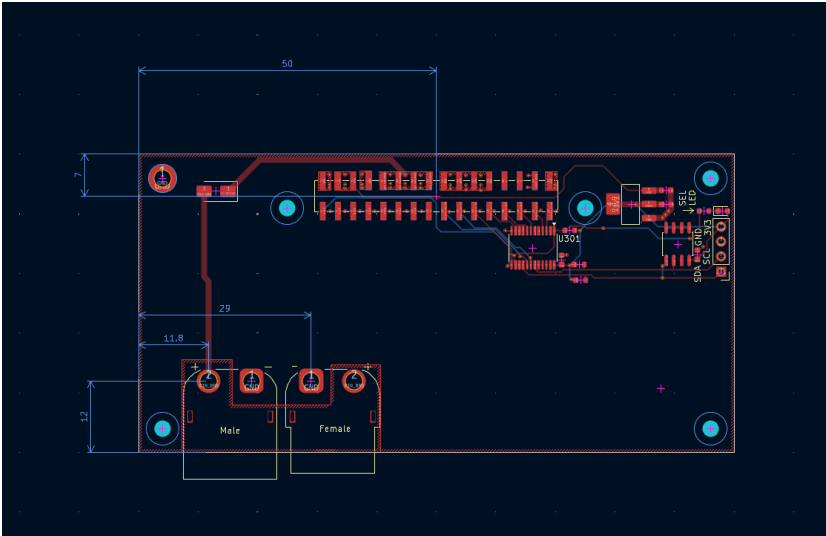


Figure 6.2: Rozstaw złącz

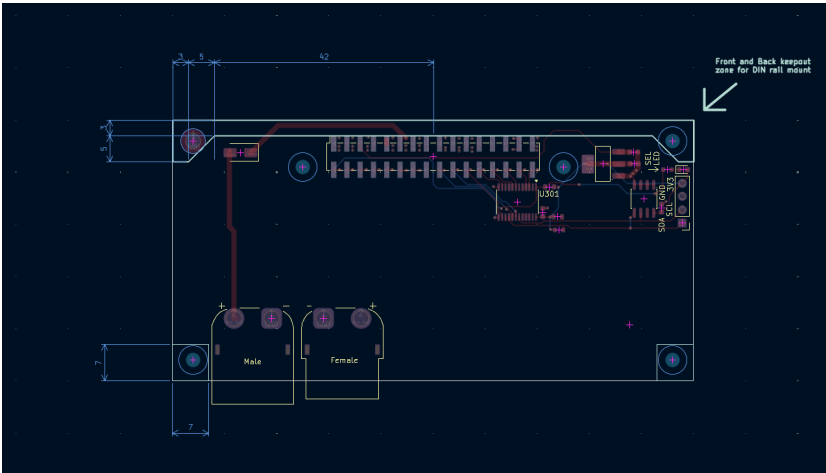


Figure 6.3: Miejsce zarezerwowane na montaż

7 Książka system integrator

8 Książka ecosystem maintainer

Okay, jako że to jest mój fork to sobie zapisze notatki co do rozwoju systemu

- Nazwa MMS3 both do systemu Kontroler + hat i też do samego controlera jest confusing
- trzeba wymyślić nazwe na hat'y który 1. nie zabierają ADDR 2. Odwalają coś z ADDR (hat PSU i hat hat expander)
- każdy projekt na githubie musi mieć tag żeby dało rade go szukać sensownie
- programowanie musi być proste, KISS it or nobody will use it
- Trzeba porządnie usiąść do licencji prawnych. Hat'y mają jakieś, ale to temp solutions